Aula 10

Análise de Algoritmos

Cris Sato

- Vimos vários algoritmos de ordenação
- Os mais eficientes com ordem $O(n \lg n)$, como o MergeSort e o HeapSort

Será possível desenvolver um algoritmo que é $o(n \lg n)$?

Depende do que sabemos sobre a entrada

- Por exemplo, suponha que saibamos que os números no vetor estão entre 0 a M

Problema: dado um vetor A[1...n] com números entre 0 a M, ordenar A

O importante é que M não faz parte da entrada, mas sim do problema

$M = 9$

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

C[0..9] : C[i] = número de vezes que i aparece em A

↳ C começa em 0

C

0	4	1	2	3	0	0	2	2	0
0	1	2	3	4	5	6	7	8	9

A	1	1	1	1	2	3	3	4	4	4	7	7	8	8
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

```
COUNTINGSORT(A, n)
C = um vetor de M posições, inicializado
    com 0 em todas as posições
para i de 1 a n:
    C[A[i]]++
i = 1
para j de 0 a M:
    para k de 1 a C[j]:
        A[i] = j
        i++
```

$\Theta(M)$

$\Theta(n)$

$\Theta(M+n)$

$\Theta(M+n)$. Como M é constante, $\Theta(n)$

LISTA 10A — Q1

Se não tivemos o valor de M fixado e, ao invés disso, calcularmos M como o valor máximo dos elementos em A, o algoritmo resultante é considerado eficiente?

- Vamos ver um outro algoritmo. Suponha que os números são naturais com no máximo D dígitos

D = 3

241	341	909	309	001	009	342	242
1	2	3	4	5	6	7	8

Ordenar pelo 1º dígito
(menos significativo)

241	341	001	342	242	909	309	009
1	2	3	4	5	6	7	8

Ordenar pelo 2º dígito

001	909	309	009	241	341	342	242
1	2	3	4	5	6	7	8

Ordenar pelo 3º dígito

001	009	241	242	309	341	342	909
1	2	3	4	5	6	7	8

FIM

001	009	241	242	309	341	342	909
1	2	3	4	5	6	7	8

```
RADIX_SORT(A,n)
  para d de 1 até D:
      ordene A pelo d-ésimo
      dígito
```

Com qual algoritmo?

- Se a ordenação por cada dígito demora tempo $\Theta(n)$, o tempo de execução é $\Theta(Dn)$
- Mas como podemos garantir que a ordenação do terceiro dígito não "embaralhou" a ordenação dos dígitos anteriores

- Suponha que já ordenamos A pelos dígitos 1, 2,...,d e que A está ordenado se considerarmos apenas esses dígitos

$d=2$

001	909	309	009	241	341	342	242
1	2	3	4	5	6	7	8

- Se a e b são dois números com a aparecendo antes de b com 3º dígitos d_a e d_b

└ $d_a < d_b$: a deve aparecer antes de b

Exemplo: a = 009 e b = 242

Ordenar pelo 3º dígito funciona

- $d_a > d_b$: b deve aparecer antes de a
  Exemplo: a = 341 e b = 242
  Ordenar pelo 3º dígito funciona

- $d_a = d_b$: a deve aparecer antes de b
  Exemplo: a = 309 e b = 341
  Na ordenação pelo 3º dígito a e b empatam e precisamos manter a antes de b

LISTA 10A — Q2

Suponha que, após ordenar pelo primeiro dígito, chegamos na seguinte configuração:

990	240	291	131	712	892
1	2	3	4	5	6

Qual deve ser a configuração após ordenar pelo segundo dígito?

Queremos que o algoritmo de ordenação auxiliar seja **<u>estável</u>**:

se a e b empatam no critério de ordenação
e a aparece antes que b no vetor,
então a ainda deve aparecer antes que b no vetor ordenado

LISTA 10A — Q3

Quais dos algoritmos de ordenação a seguir são estáveis?

SelectionSort

InsertionSort

MergeSort

HeapSort

QuickSort

- No entanto, nenhum desses faz sentido aqui. Se utilizarmos um deles, é mais fácil ordenar o vetor de uma vez ao invés de ir pelos dígitos
- Precisamos de um algoritmo que irá tirar proveito do fato que estamos ordenando pelos dígitos e que temos apenas 10 possíveis dígitos: 0–9

CountingSort?

```
para j de 1 a M:
    para k de 1 a C[j]:
        A[i] = j
        i++
```

- O CountingSort do jeito que fizemos não permite que A guarde mais informações do que o número a ser ordenado. Pois não copia A[i] apenas preenche com j
- Ou seja, não permite informação satélite. O que é muito comum de querermos em aplicações

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$C[1..10]$: $C[i]$ = número de vezes que i aparece em A

C	0	4	1	2	3	0	0	2	2	0
	0	1	2	3	4	5	6	7	8	9

somas parciais

C	0	4	5	7	10	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

C	0	4	5	7	10	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

Onde colocar A[14] = 1?

C[1] = 4, então temos 4 elementos ≤ 1. Como queremos manter estável e A[14] está na última posição, precisamos colocar A[14] o mais "para frente" possível

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

C	0	4	5	7	10	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

B				1										
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

será o vetor de saída

$B[4] \leftarrow A[14]$

C	0	3	5	7	10	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

C	0	3	5	7	10	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

B				1						4				
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$B[10] \leftarrow A[13]$

C	0	3	5	7	9	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

C	0	3	5	7	9	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

B				1			3			4				
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$B[10] \leftarrow A[13]$

C	0	3	5	6	9	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

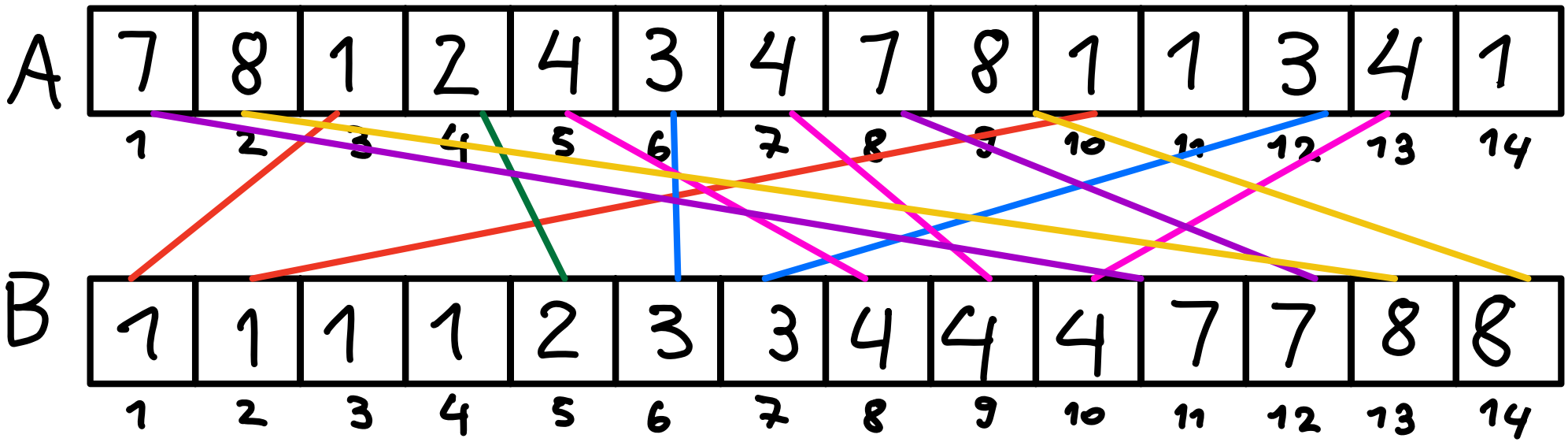
A	7	8	1	2	4	3	4	7	8	1	1	3	4	1
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

C	0	3	5	6	9	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9

B			1	1			3			4				
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$B[10] \leftarrow A[13]$

C	0	2	5	6	9	10	10	12	14	0
	0	1	2	3	4	5	6	7	8	9



j

$v = A[j]$

$p = C[v]$

$$\begin{cases} B[p] \leftarrow A[j] \\ C[v]{-}{-} \end{cases}$$

```
COUNTING SORT_ESTÁVEL(A,B ,n)
C = um vetor de M posições, inicializado
    com 0 em todas as posições        Θ(M)
para i de 1 a n:                      Θ(n)
    C[A[i]]++
para v de 1 a M:                      Θ(M)
    C[v] = C[v-1] + C[v]
para j de n a 1:                      Θ(n)
    B[C[A[j]]] = A[j]
    C[A[j]]--
```

$\Theta(n+M)$

```
RADIX_SORT(A,n)
  para d de 1 até D:
      ordene A pelo d-ésimo
      dígito utilizando o
      COUNTINGSORT_ESTÁVEL
```

aqui adapte para utilizar o d-ésimo dígito

$\boxed{\Theta(Dn)}$

LISTA 10A — Q4

Execute o passo com j= 10

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
A	7	8	1	2	4	3	4	7	8	1	1	3	4	1

	0	1	2	3	4	5	6	7	8	9
C	0	2	5	6	9	10	10	12	14	0

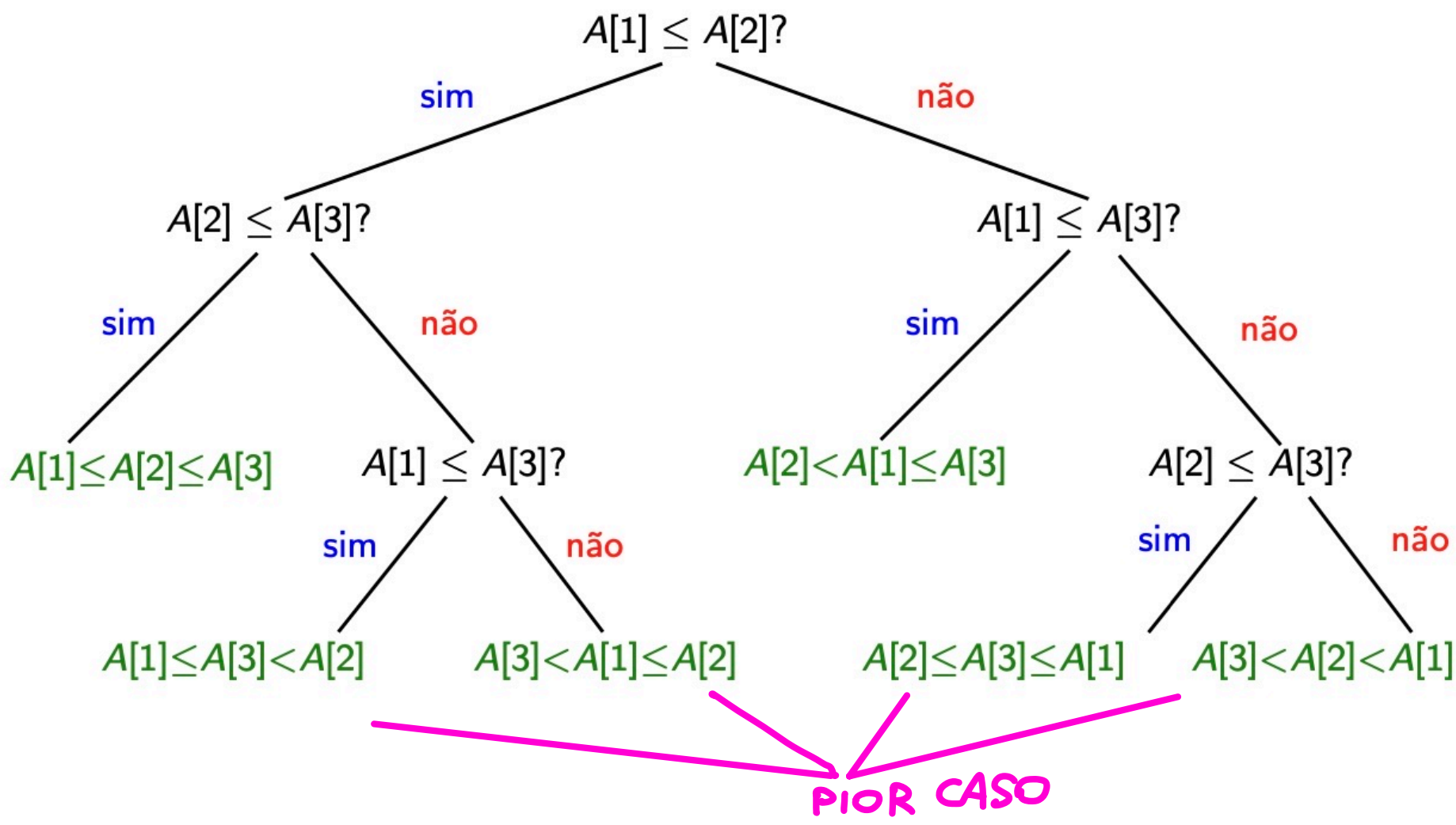
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
B			1	1			3			4				

Considere o problema de ordenação em que não temos informação adicional sobre a entrada

Será possível desenvolver um algoritmo que é $O(n \lg n)$?

Suponha que seu algoritmo é baseado em <u>comparações</u>, ou seja, sabemos decidir se $a < b$, $a > b$ ou $a = b$. Veremos que neste caso a resposta é <u>NÃO</u>

Considere como o INSERTION SORT ordena $A[1..3]$:



- No pior caso, o número de comparações realizadas é a altura da árvore
- Ao final da ordenação, estamos em uma folha
- Cada algoritmo de ordenação implementa uma árvore binária como esta
- O número de possíveis configurações de A é n!
- Cada configuração corresponde a uma folha
- Mas toda árvore binária de altura h tem no máximo 2^h folhas (fácil de provar)
- Então para conseguirmos n! folhas, qual é a altura mínima da árvore binária?

$$n! \geq \left(\frac{n}{e}\right)^n$$

vem do fato que a aproximação de Stirling $n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$ satisfaz $n! \geq \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

$$\left(\frac{n}{e}\right)^n \leq 2^h \Rightarrow \left(\frac{n}{e}\right)^n \leq e^h \Rightarrow \ln\left(\frac{n}{e}\right)^n \leq \ln(e^h)$$

$$\Rightarrow n\ln\left(\frac{n}{e}\right) \leq h \Rightarrow h \geq n\ln n - n$$

Ou seja, $\boxed{h = \Omega(n\ln n)}$

Todo algoritmo de ordenação
baseado em comparações faz

$$\Omega(n \lg n)$$

comparações no pior caso.

LISTA 10A — Q5

Digamos que Bob rodou um algoritmo de ordenação baseado em comparações para diversos vetores que ele gerou aleatoriamente.

O número máximo de comparações que encontrou parecia ser linear em n.

Isso é possível?